

RAPIZZA Ltda.

Documentación de Arquitectura del Sistema

Programación II — Cliente / Servidor
Universidad Libre Seccional Pereira · Facultad de Ingeniería · 2026-1

Proyecto: Sistema de Gestión de Pedidos RAPIZZA Ltda.

Stack: React 18 · Node.js · Express · MySQL 8

Seguridad: bcrypt 10 rounds · JWT (8h) · RBAC por rol

Valor: 40% de la nota final — Segundo Corte

1. Arquitectura de Tres Capas

La aplicación sigue el patrón **Cliente/Servidor** con separación estricta de responsabilidades en tres capas independientes. Esta arquitectura es el estándar de la industria para aplicaciones web empresariales modernas.

Capa	Tecnología	Responsabilidad	Puerto
Presentación (Frontend)	React 18 + Vite TailwindCSS + Axios	Interfaz de usuario, formularios, navegación y consumo de la API REST	5173
Lógica de Negocio (Backend)	Node.js + Express bcrypt + JWT	API RESTful, validaciones, autenticación, reglas de negocio y control de roles	3001
Persistencia (Base de datos)	MySQL 8 Normalización 3FN	Almacenamiento relacional de pedidos, clientes, empleados y transacciones	3306

Flujo de comunicación

Cada petición sigue un ciclo definido: el usuario interactúa con React → Axios envía la petición HTTP con el JWT en el header Authorization → Express valida el token con el middleware verifyToken → el controlador ejecuta la consulta SQL → MySQL retorna los datos → Express responde con JSON estructurado { success, data, message } → React actualiza el estado y re-renderiza la UI.

#	Quién actúa	Acción
1	Usuario (navegador)	Hace clic o llena un formulario en React
2	React + Axios	Envía petición HTTP con Bearer token
3	Express (server.js)	Recibe la petición, aplica CORS y parsea JSON
4	verifyToken.js	Verifica y decodifica el JWT
5	checkRole.js	Valida que el rol tenga permiso para esa ruta
6	Controlador	Ejecuta lógica de negocio y consulta MySQL
7	MySQL	Retorna resultados al controlador
8	Express	Responde res.json({ success, data, message })
9	React	Actualiza estado y re-renderiza la interfaz

2. Descripción de Componentes

Frontend — /frontend/src/

Archivo / Módulo	Responsabilidad
pages/Login.jsx	Formulario de autenticación. Llama a authService.login() y guarda el JWT en localStorage.
pages/Dashboard.jsx	Hub de navegación post-login. Muestra estadísticas en tiempo real y menú filtrado por rol.
pages/Pedidos.jsx	Módulo principal. Permite crear pedidos, asignar repartidores y registrar entregas con cálculo automático de tardanza.
pages/Clientes.jsx	CRUD de clientes. El despachador puede registrar y editar; la búsqueda filtra por nombre, cédula y teléfono.
pages/Productos.jsx	CRUD de pizzas y aditivos (solo admin). Usa soft-delete para preservar el historial de pedidos.
pages/Empleados.jsx	Gestión del personal. Muestra conteo por tipo (chef, despachador, repartidor) y permite asignar cuentas.
pages/CuentasCobro.jsx	Lista cuentas generadas automáticamente por pedidos tardíos. El despachador las marca como pagadas.
context/AuthContext.jsx	Estado global de sesión con React Context. Provee user, token, login() y logout() a toda la app.
services/api.js	Instancia Axios con interceptor que adjunta el JWT en cada petición y redirige al login si el token expira.
components/PrivateRoute	HOC que protege rutas. Verifica sesión activa y opcionalmente el rol antes de renderizar.

Backend — /backend/

Archivo / Módulo	Responsabilidad
server.js	Punto de entrada. Configura Express, CORS, middlewares globales y monta todas las rutas API.
config/db.js	Pool de conexiones MySQL con mysql2/promise. Verifica la conexión al arrancar.
middleware/verifyToken.js	Extrae y valida el JWT del header Authorization. Inyecta req.user = { id, rol }.
middleware/checkRole.js	Fábrica de middlewares. checkRole('admin') permite solo admins; acepta múltiples roles.
controllers/authController	Register (bcrypt hash), login (bcrypt.compare + jwt.sign) y endpoint /me para verificar sesión.
controllers/pedidoController	Lógica de negocio central: crea pedidos en transacción SQL, calcula estado al entregar (> 30 min → cuenta_cobro) y genera cuentas de cobro automáticamente.
controllers/rapizzaController	CRUD de clientes, productos (soft-delete), empleados y cuentas de cobro.

routes/	Un archivo por recurso. Define método HTTP, URL, middlewares de auth/rol y controlador.
----------------	---

3. Modelo de Datos

La base de datos **rapizza_db** está normalizada en Tercera Forma Normal (3FN). Consta de 7 tablas con relaciones bien definidas y restricciones de integridad referencial.

Tabla	PK	Relaciones	Descripción
usuarios	id (INT AI)	1:1 → empleados	Cuentas de sistema con hash bcrypt y rol ENUM.
empleados	id_empleador (INT AI)	1:1 ← usuarios 1:N → pedidos (x3)	Personal: chef, repartidor, despachador.
clientes	cedula (VARCHAR 20)	1:N → pedidos	Datos de contacto y zona de cobertura.
productos	codigo (VARCHAR 20)	N:M → pedidos via detalle_pedido	Pizzas y aditivos con precio. Soft-delete.
pedidos	numero (INT AI)	N:1 ← clientes N:1 ← empleados (x3) 1:N → detalle_pedido 1:1 → cuentas_cobro	Eje central del sistema. Estado automático al entrega
detalle_pedido	id (INT AI)	N:1 ← pedidos N:1 ← productos	Tabla intermedia N:M. Guarda precio al momento del
cuentas_cobro	id (INT AI)	1:1 ← pedidos N:1 ← empleados	Generada automáticamente cuando entrega > 30 min

Lógica de negocio en la capa de datos

Al registrar la **hora_entrega** de un pedido, el backend calcula los minutos transcurridos desde hora_salida. Si supera los 30 minutos, el estado_pago cambia a **cuenta_cobro** y se inserta automáticamente un registro en cuentas_cobro vinculado al repartidor responsable. Si llega a tiempo, el estado es **pagado**. Esta lógica está encapsulada en el controlador con una transacción SQL para garantizar atomicidad.

4. Endpoints de la API REST

Paragr aph('c aseSe nsitive' : 1 'en coding ' : 'utf8' 'text': ' AUTE NTIC ACIÓ N 'frags': [ParaF rag(___ tag___ ='b', b old=1, fontNa me='H elvetic a-Bold ' , font Size= 10, gr eek=0, italic= 0, link=[], rise=0, text=' AUTE NTICA CIÓN', textCo lor=Co lor(0,0 ,0,1), us_lin es=[])] 'style': 'bullet Text': None ' debug' : 0) # Paragr aph				
---	--	--	--	--

Método	Endpoint	Body (JSON)	Respuesta	Auth
POST	/api/auth/register	{ usuario, password, rol }	{ success, data: { id, usuario, rol } }	No
POST	/api/auth/login	{ usuario, password }	{ success, data: { token, usuario, rol, imagen } }	No
GET	/api/auth/me	—	{ success, data: { id, usuario, rol, imagen } }	JWT

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'PEDIDOS''frags': [Paragraph(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='PEDIDOS', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>				
Método	Endpoint	Body (JSON)	Respuesta	Rol
GET	/api/pedidos	—	Lista de pedidos con joins	JWT

GET	/api/pedidos/:id	—	Pedido + detalle completo	JWT
POST	/api/pedidos	{ fecha, hora_salida, cedula_cliente, detalle[] }	Pedido creado + total	despachador
PUT	/api/pedidos/:id/asignar-repartidor	{ id_repartidor }	Confirmación	despachador
PUT	/api/pedidos/:id/entregar	{ hora_entrega }	Estado calculado + minutos	repartidor
DELETE	/api/pedidos/:id	—	Confirmación	admin
GET	/api/pedidos/estadisticas	—	{ total, pendientes, tardios, perdidas }	despachador

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DOMINIO' 'frags': [Paragraph(__tag__ = 'b', bold=1, fontName='Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='DOMINIO', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>				
Método	Endpoint	Body (JSON)	Respuesta	Rol
GET	/api/clientes	—	Lista de clientes	JWT

POST	/api/clientes	{ cedula, nombre, telefono, direccion, zona_cobertura }	Cliente creado	despachador
PUT	/api/clientes/:cedula	{ nombre, telefono, direccion, zona_cobertura }	Confirmación	despachador
GET	/api/productos	—	Productos activos	JWT
POST	/api/productos	{ codigo, nombre, valor_unitario }	Producto creado	admin
PUT	/api/productos/:codigo	{ nombre, valor_unitario }	Confirmación	admin
DELETE	/api/productos/:codigo	—	Soft-delete	admin
GET	/api/empleados	?tipo=repartidor	Lista filtrada	JWT
POST	/api/empleados	{ nombre, tipo }	Empleado creado	admin
PUT	/api/empleados/:id	{ nombre, tipo, activo }	Confirmación	admin
GET	/api/cuentas-cobro	—	Cuentas con repartidor y pedido	despachador
PUT	/api/cuentas-cobro/:id/pagar	—	Confirmación	despachador

5. Seguridad y Control de Acceso

5.1 Autenticación con bcrypt + JWT

Las contraseñas nunca se almacenan en texto plano. Al registrar un usuario, se aplica **bcrypt con 10 rounds** (factor de costo). Al iniciar sesión, `bcrypt.compare()` verifica el hash sin descriptarlo. Si las credenciales son correctas, el backend genera un **JWT firmado con HS256** que expira en 8 horas. El token se guarda en `localStorage` del navegador y se adjunta en cada petición mediante el header *Authorization: Bearer <token>*.

5.2 Control de acceso por rol (RBAC)

Rol	Permisos
admin	Acceso total: CRUD de usuarios, empleados, productos. Ve estadísticas y cuentas de cobro.
despachador	Crea pedidos, asigna repartidores, registra clientes, gestiona cuentas de cobro.
chef	Acceso de solo lectura al dashboard y lista de pedidos asignados.
repartidor	Solo puede registrar la hora_entrega de pedidos que tiene asignados.

5.3 Variables de entorno

Las credenciales de la base de datos y el `JWT_SECRET` se configuran en el archivo **.env** que nunca se sube al repositorio (está en `.gitignore`). El repositorio incluye un archivo **.env.example** con los nombres de las variables pero sin valores reales, para que cualquier desarrollador pueda configurar su entorno.

6. Justificación de Decisiones Técnicas

Decisión	Justificación
mysql2 directo en lugar de Sequelize	Para proyectos de este tamaño, el driver nativo permite mayor control sobre las queries SQL, fa
Transacción SQL al crear pedidos	La creación de un pedido involucra múltiples inserciones (pedidos + N filas de detalle_pedido). S
Lógica de 30 minutos en el backend	La regla de negocio (> 30 min = cuenta de cobro) se implementa en el servidor, no en el fronten
Soft-delete en productos	Eliminar físicamente un producto rompería las referencias históricas en detalle_pedido (con ON
React Context para la sesión	Alternativas como Redux serían sobredimensionadas para este proyecto. Context API es nativa
Proxy de Vite hacia el backend	El archivo vite.config.js redirige /api al puerto 3001, evitando errores de CORS en desarrollo y p
SweetAlert2 para confirmaciones	El enunciado lo exige explícitamente (RF-04) y prohíbe alert() nativo. SweetAlert2 permite perso